

Smart Hospital Patient and Resource Allocation System Project Report

CSC 1012 – Introduction to Computer Programming
Faculty of Applied Sciences, University of Sri Jayewardenepura

Name	R.A.L.M. Ranatunga
Index No	AS20250450
GitHub Repository	github.com/lochanalmr/smart-hospital-patient-and-resource-allocation-system

Contents

High Level Architecture	3
Overview	3
List of functions in the program.....	3
Data structure design	4
Function implementation details	5
1 – Patient registration.....	5
2 – Estimated waiting time calculation	5
3 – Emergency surcharge calculation.....	5
4 – Ward stay cost calculation.....	6
5 – Total bill calculation	6
6 – Age subsidy discount calculation	6
7 – Find free bed	6
8 – Display patient bill	6
9 – Return triage level text.....	6
10 – Return surcharge percentage.....	7
11 – Sort patients by priority	7
12 – Display patients by priority	7
13 – Generate and display performance reports.....	7
14 – Load bed occupancy.....	8
15 – Save bed occupancy	8
16 – Add patient record to file	8
Assumptions made during development.....	9
Limitations of the program	9
Captures from program runtime.....	10
Commit history.....	12

High Level Architecture

Overview

The program is a menu-driven Smart Hospital Patient and Resource Allocation system, where patients can be registered and billed. ([link to GitHub repository](#))

The bill is created using the data entered at registration, and by doing various calculations on the provided data.

In addition to that, a list of existing registered patients sorted according to urgency level and registration order, can be viewed.

Only source code files (.c and .h files) were pushed to GitHub, by using a **.gitignore** file. (The **.gitignore** file was originally created using a template provided by GitHub, at the time of creating the repository, and was later modified, to also ignore codeblocks generated files and folders)

README.md contains a summary about the project, and details to compile and use the program outside of codeblocks IDE.

List of functions in the program

- Patient registration (**patientRegistration**)
- Estimated waiting time calculation (**calculateEstimatedWaitingTime**)
- Emergency surcharge calculation (**calculateEmergencySurcharge**)
- Ward stay cost calculation (**calculateWardStayCost**)
- Total bill calculation (**calculateTotalBill**)
- Age subsidy discount calculation (**calculateAgeSubsidyDiscount**)
- Find the first free bed in a ward (**findFreeBed**)
- Display bill of a patient (**displayPatientBill**)
- Return triage level text (**triageLevelText**)
- Return surcharge percentage (**surchargePercentage**)
- Sort patients by priority (**sortPatientsByPriority**)
- Display patients by priority (**displayPatientsByPriority**)
- Generate and display performance reports (**generatePerformanceReport**)
- Load bed occupancy (**loadBedOccupancy**)
- Save bed occupancy (**saveBedOccupancy**)
- Add patient record to file (**addPatientRecord**)

Data structure design

Parallel arrays were used for storing data, and it is the data structure design used in this program.

Each type of array is parallel to each other. (doctor specialties data arrays and hospital data arrays, patient data arrays)

Patient data arrays were implemented inside the **main.c** file, while doctor specialties data arrays, patient data arrays and bed tracking array was implemented inside a separate **data_arrays.c** file.

For example, the arrays for doctor specialties data, are implemented in the following way:

```
const char *specialtyNames[] = {  
    "General Practice (OPD)", "Pediatrics", "Cardiology", "Neurology"  
};
```

```
const int baseConsultationFees[] = {  
    1500, 2500, 4500, 5000  
};
```

```
const int consultationTimesPerPatient[] = {  
    15, 20, 30, 30  
};
```

```
const int dailyPatientCaps[] = {  
    30, 20, 12, 10  
};
```

So, if we access index 0 of any of these arrays, it will correspond to the data for “General Practice (OPD)” data.

Likewise, hospital data arrays are also implemented in the same manner.

Additionally, an array was implemented to track the bed using 1/0 status, where 0 is there by default, and set to 1 when a bed of a ward is booked, using a function.

Patient data is stored using parallel arrays in **main.c**, with one array per attribute, and indexed by the same patient number, so the patient registration function can be called with the addresses of each array element.

Function implementation details

1 – Patient registration

This function collects patient data required for billing purposes.

The arrays required for storing patient data are implemented inside `main.c`, and then the elements of arrays for each patient are passed by their addresses.

The function writes each attribute using **scanf** function, and each attribute is validated using a do-while loop.

As an example:

```
do{
    printf("Enter patient age: ");
    scanf("%d", age);
} while(*age <= 0);
```

The function returns 0 if the patient was successfully registered, or 1 if the patient could not be registered due to reaching maximum capacity of the ward or specialty.

72 patients can be accepted maximum per program run, when patients for all specialties are filled up.

The function allocates a bed according to their admitted ward in the bed tracking array (implemented inside `data_arrays.c`), when a patient is admitted to a ward.

2 – Estimated waiting time calculation

This function calculates the estimated waiting time using the patients in queue for that specialty, and the consultation time per patient for that specialty.

As this function is called by the patient registration function, after the count of patients for each specialty is increased, when getting the no. of patients before the current patient, we need to reduce 1 from the count.

```
peopleBeforePatient = countPerSpecialty[specialtyOfPatient - 1] - 1;
```

Waiting time is calculated using the following equation:

```
waitTime = peopleBeforePatient *
consultationTimesPerPatient[specialtyOfPatient - 1];
```

3 – Emergency surcharge calculation

Using the triage level given by the user, base consultation fee is retrieved.

To retrieve the base consultation fee, 1 is reduced from the specialty ID when accessing the array, as the specialty ID array index starts from 0. (User gives 1-4, array has 0-3)

Using the base consultation fee and triage level, emergency surcharge is calculated.

4 – Ward stay cost calculation

Ward stay cost is calculated using admitted days, and daily rate for the bed in the specified ward.

When accessing the daily bed rate array, ward ID - 1 is accessed, as the array index starts from 0.

5 – Total bill calculation

This function returns the total bill value, which consists of base consultation fee, surcharge and ward stay cost.

6 – Age subsidy discount calculation

Age is provided to the function to determine whether the patient is eligible for the discount, and if yes, it returns the discount value, otherwise returns 0.

7 – Find free bed

The bed tracking array, ward ID of the patient and total bed capacity array is provided to the function.

The first bed of the relevant ward, with bed tracking status set to 0 will be found, and that bed number will be returned.

If no free bed is found, function will return -1, to signal the patient registration function that a new patient cannot be registered as there are no beds available in the specified ward.

8 – Display patient bill

All details taken during registration and all other calculated information of the patient that the bill needs to be displayed is passed into the function.

The bill is designed according to the specified template given in the requirements.

9 – Return triage level text

This function returns the text required for the bill by using the triage level integer.

10 – Return surcharge percentage

This function returns the surcharge percentage according to their triage level. Only used internally for the bill displaying function.

11 – Sort patients by priority

This function uses selection sort to sort patients by priority and registration order.

It sorts patients so that the highest triage level comes first. If two patients have the same triage level, the one who registered earlier comes first.

The algorithm works like this:

- First, it stores each patient's original position in the **order[]** array.
- Then, it checks the patients one by one and finds the patient with the highest triage level.
- If patients have the same triage level, it chooses the patient with the smaller original index.
- Finally, it swaps the positions in **order[]** to put the patients in the correct order, without changing the original **triageLevels[]** array.

12 – Display patients by priority

The sorted list of patients is displayed here, along with important details about the patient, like the name, age, triage level, specialty, ward, bed number and waiting time.

Ward and bed number are only shown when admitted to a ward.

13 – Generate and display performance reports

This function generates a summary report of hospital operations, using the data of all registered patients.

If no patients are registered, the function displays a message and returns.

Otherwise, it loops through all patients once, to count patients per triage level, to get the total revenue and the total discounts, and find the index of the highest-paying patient by comparing total bill values.

Using the collected data, the report is displayed in 4 sections. (patients registered and categorized by urgency level, total revenue and total discounts, bed occupancy percentage per ward, and details of the highest-paying patient)

14 – Load bed occupancy

This function reads the saved bed statuses from **beds_status.txt** and stores them in the bed tracking array. It also counts the occupied beds in each ward. It returns 0 when the file is not available or is loaded successfully, and 1 if the file contains invalid data.

15 – Save bed occupancy

This function writes the current bed tracking array to **beds_status.txt**. The file is overwritten so that it contains the latest bed status for every ward. It returns 0 when the file is saved successfully, or 1 if the file cannot be opened.

The function writes the patient number, so if we need to free up a bed on the next day, we can manually set that bed to 0 by editing the text file.

16 – Add patient record to file

This function adds the patient's ID, name, age, bill, discount, and final payable amount to **patient_records.txt**. The file is opened in append mode so previous patient records are kept. It returns 0 when the record is saved successfully, or 1 if the file cannot be opened.

The patient records will not be able be used again, and only for saving historical data.

Assumptions made during development

Below are some assumptions that were done to simplify the development of the program.

1. Maximum length of a patient's name does not exceed 49 characters
2. User is compliant to the requests, and enter data only in the requested data type (program checks for invalid inputs in the same data type, like whether age is larger than 0, but will crash if a string is given)
3. User will index patient, ward or specialty indexes from 1 (the program assumes that the user will enter 1 for the 0th index in the array, and is designed to reduce 1 from user input in all instances that searches information from an array)
4. The no. of days that a patient is admitted (or whether admission is not required) is known at registration, as the system asks for information about ward admission, and no. of days admitted, at registration.

Limitations of the program

1. Bed tracking text file needs to be edited manually when a patient's bed is no longer occupied
2. Program cannot load historical data from patient data text file, and is only available for viewing purpose.
3. Cannot accommodate new patients for patients leaving during the day, as restarting program will cause patient data in memory to be deleted. (The user can accommodate new patients by manually setting bed status to 0 in the bed tracking status text file, but needs to restart the program. When restarted, as the program does not contain the previous records in the day, sorted patient lists and reports generated by the program will be of no use as they do not contain previous data during the day)
4. Patient data cannot be edited or deleted once a patient is registered

Captures from program runtime

1 – Menu

Smart Hospital Patient and Resource Allocation System

Main menu:

1. Register patient
2. View bill of a patient
3. View sorted list of patients
4. View report and analytics
5. Exit program

Enter choice: |

2 – Patient registration

---Patient Registration---

1 - Patient Details Collection

Enter patient name: Lochana

Enter patient age: 21

Emergency triage level (1 = Normal, 2 = Urgent, 3 = Critical): 1

2 - Specialty selection

Specialty ID (1 - 4): 2

3 - Ward Admission Details

Is admitted to ward? (1 = Yes, 0 = No): 1

Enter ward ID (1 to 4): 3

Enter days admitted: 5

Patient 1 successfully registered!

3 – Viewing bill

Enter Patient ID to display bill: 1

=====

SMART HOSPITAL ADMISSION & BILL

Patient ID : PAT-0001
Patient Name : Lochana
Age : 21 Years (Subsidy Not Eligible)
Specialty : Pediatrics
Assigned Ward : Surgical Ward (Bed #01)
Urgency Level : Level 1 (Normal)

Base Consultation Fee : LKR 2500.00
Emergency Surcharge : LKR 0.00 (0%)
Ward Stay Cost (5 Days) : LKR 60000.00

Gross Total Bill : LKR 62500.00
Age Subsidy Discount : LKR -0.00 (0%)

Final Payable Amount : LKR 62500.00
Estimated Waiting Time : 0 mins (Immediate Attention)
=====

4 – Viewing sorted list of patients

-----Displaying sorted list of patients-----

Priority	Patient ID	Name	Age	Urgency	Specialty	Ward	Waiting Time (min)
1	PAT-0005	Daham	21	Level 3 (Critical)	General Practice (OPD)	W3-B03	15
2	PAT-0001	Lochana	21	Level 1 (Normal)	Pediatrics	W3-B01	0
3	PAT-0002	Dulwin	19	Level 1 (Normal)	Cardiology	W2-B01	0
4	PAT-0003	Linuka	21	Level 1 (Normal)	Neurology	W2-B02	0
5	PAT-0004	Kavisha	20	Level 1 (Normal)	General Practice (OPD)	W3-B02	0

5 – Viewing reports and analytics

1. Patients Registered by Urgency Level

Total Patients Registered : 5
Level 1 (Normal) : 4
Level 2 (Urgent) : 0
Level 3 (Critical) : 1

2. Revenue & Discounts

Total Revenue Earned : LKR 201750.00
Total Discounts Granted : LKR 0.00

3. Bed Occupancy Percentage Per Ward

General Ward : 0.00% (0/20 beds occupied)
Pediatric Ward : 20.00% (2/10 beds occupied)
Surgical Ward : 30.00% (3/10 beds occupied)
ICU (Intensive Care Unit) : 0.00% (0/5 beds occupied)

4. Highest-Paying Patient

Patient Name : Lochana
Patient ID : PAT-0001
Total Bill : LKR 62500.00

Commit history

The commit history is listed from latest to earliest, using git log command

- 2b08517 2026-09-19 Add ability to prevent registering new patients when all beds of the ward are occupied
- 57190af 2026-09-19 Fix errors in logic
- 90db33b 2026-09-19 Add file handling to program
- 8e91574 2026-09-19 Fix minor issues in overall code, and ability to enter name with spaces introduced
- d85e5b1 2026-09-17 Update README.md
- 92de6fc 2026-09-17 Fix logical errors and formatting issues
- 9bd4d42 2026-09-17 Add functionality to view reports
- c46adf9 2026-09-17 Implement viewing sorted list of patients
- 7211e1d 2026-09-17 Implement display bill functionality
- 0b17230 2026-09-17 Add bed status tracking to the patient registration function
- 6b293dd 2026-09-13 Corrected function names
- 4956e3e 2026-08-25 Add final bill calculation in patient registration function
- d41aee1 2026-08-25 Implement age subsidy discount calculation
- 385b5d3 2026-08-25 Implement total bill calculation functionality
- f29d677 2026-08-25 Implement total ward cost calculation functionality
- 2dc0ad4 2026-08-24 Add emergency surcharge calculation function
- 0ddd304 2026-08-24 Make changes to accommodate new functionality
- 5949d3c 2026-08-24 Create header file for patient_registration.c
- 5a7470a 2026-08-24 Add estimated waiting time calculation feature
- b2c2830 2026-08-24 Create header file for calculations.c
- e58bbe0 2026-08-24 Implement estimated waiting time calculating function
- b78d8fc 2026-08-24 Create header file for data arrays
- 095c0d1 2026-08-23 Implement arrays for storing registration data
- d4feda0 2026-07-31 Link patient registration function to main.c
- c485422 2026-07-31 Implement patient registration functionality
- 347fa40 2026-07-30 Implement arrays required for the program
- f51f81d 2026-07-30 Add codeblocks folders and files to .gitignore
- 3c4745f 2026-07-30 Initial commit

